

Scaling Neural Network Verification with Tensor Parallelism and Fully Sharded Data Parallelism

Sergei Vorobyov¹  and Eugene Ilyushin^{1,2} 

¹ Lomonosov Moscow State University, Moscow, Russia

`sergei.vorobyov01@gmail.com`

² Central University, Moscow, Russia

`eugene.ilyushin@gmail.com`

Abstract. Formal neural network verification is bounded in practice by GPU memory, and it is widely assumed that the weight matrices are what has to be distributed. We test that assumption by porting two sharding strategies from large-scale model training to `auto_LiRPA` / α, β -CROWN and by measuring, rather than estimating, where the memory goes. *Tensor Parallelism* shards the weights together with the CROWN A -matrices, i.e. the specification axis: peak memory falls by $1.97\times$ at $P=2$ and $3.83\times$ at $P=4$ and propagation becomes $1.90\times$ and $3.49\times$ faster. Its bounds are exact for the first sharded zone. Later zones fall back to IBP, and the bounds get looser with each of them. *Fully Sharded Data Parallelism* shards only the weights. Its bounds are bitwise identical to the single-GPU ones, but peak memory goes *up* by 26–34% and propagation takes up to $3.2\times$ longer, mostly in collectives. Simply staging the weights from host memory is better on both counts. A per-node trace shows why: A -matrices are $5.6\times$ the weights already at $h=4096$, and in β -CROWN+Branch-and-Bound peak memory is linear in the number of subdomains, $21.3 + 0.372 B$ MB, of which weights are 0.13%. Sharding the subdomain axis is therefore the direction that pays, and we quantify it: a projected 45.0% and 67.5% less memory per GPU at $P=2$ and $P=4$.

Keywords: Neural network verification · Tensor parallelism · Fully sharded data parallelism · Bound propagation · α, β -CROWN · GPU memory.

1 Introduction

Neural networks are now used in vehicle control, avionics and medical decision support, where a wrong output can cause real harm. Testing is not enough there, since each test checks a single input and not the whole set of admissible inputs [8]. Formal verification proves that a property holds for *every* input in a region.

A network computes $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, and we want a property $\psi(f(\mathbf{x}))$ to hold for every input $\mathbf{x}$ in a set $\mathcal{X}$. Solvers search $\mathcal{X}$ for an input that breaks ψ . They answer SAT with such a counterexample, or UNSAT when none exists and the property holds [12]. Most benchmarks check local robustness. Here every $\mathbf{x}$ with $\|\mathbf{x} - \mathbf{x}_0\|_p \leq \varepsilon$ around a sample $\mathbf{x}_0$ of class c must also be classified as c [11].

Control tasks such as ACAS Xu use specifications over physical variables [6]. Networks and properties are exchanged in ONNX and the VNN-LIB format [5], which the VNN-COMP competition is built on [15,4,3]. The problem is NP-complete already for one hidden ReLU layer [25], $\exists\mathbb{R}$ -complete for smooth activations [2] and undecidable for some recurrent networks [17]. A *complete* verifier therefore needs Branch-and-Bound, which is exponential in the worst case. An *incomplete* one uses a polynomial relaxation instead.

1.1 The Memory Bottleneck and the Question We Ask

In practice most verifiers use *bound propagation*. The best known of them, α, β -CROWN [27], won VNN-COMP every year from 2021 to 2024. It runs on a GPU and computes linear bounds on the network output. Everything it touches has to fit into the memory of one GPU. The weights of a single hidden layer with 262 144 neurons and 4096 inputs already take about 4 GB in float32.

Parameter sharding is the reflex borrowed from large-model training, and it presupposes that parameters dominate. This paper implements that reflex faithfully in a state-of-the-art verifier, measures it, and shows the presupposition to be wrong for verification: the memory sits in the tensors indexed by the specification and subdomain axes. That is a statement about where multi-GPU verification research should aim.

Contributions.

1. Section 4 traces the memory of a CROWN pass node by node. For β -CROWN+BaB a sweep over the batch size fits $M_{\text{peak}}(B) \approx 21.3 + 0.372 B$ MB. Weights take only 0.13% of this peak.
2. Our TP version of `auto_LirPA` uses custom `BoundLinearTP_Col/Row` operators (Sections 3.2, 5.1). With 2 GPUs peak memory is $1.97\times$ lower and propagation $1.90\times$ faster; with 4 GPUs the numbers are $3.83\times$ and $3.49\times$. We also measure how much tightness IBP costs in each zone and check soundness for CROWN and α -CROWN.
3. FSDP took about 100 lines on top of `auto_LirPA`, and the same code handles convolutions and a ViT. Its bounds match the single-GPU ones bit for bit. The problem is memory. At $h=4096$ the peak is 27% higher than on one GPU, and even plain host-memory staging needs less (Sections 3.3, 5.4).
4. We give a target for future work. All batch-dependent terms grow linearly with the number of subdomains, so splitting them over P GPUs should cut memory per GPU by 45.0% at $P=2$ and 67.5% at $P=4$, or fit P times more subdomains per round (Section 5.5).

Our implementation, a fork of `auto_LirPA` / α, β -CROWN, with all experiment scripts and the collated measurement records, is publicly available.³

³ <https://github.com/vorobyov01/damdid-2026-submission>

2 Background

2.1 Bound-Propagation Methods

IBP [28] propagates intervals layer by layer: for $\mathbf{y} = W\mathbf{x} + \mathbf{b}$ it gives $\mathbf{l}_y = W^+\mathbf{l}_x + W^-\mathbf{u}_x + \mathbf{b}$ and $\mathbf{u}_y = W^+\mathbf{u}_x + W^-\mathbf{l}_x + \mathbf{b}$, with $W^\pm$ the positive and negative parts of W . It is cheap and loses tightness fast with depth. *CROWN* [28] instead relaxes each unstable ReLU with pre-activation $x \in [l, u]$, $l < 0 < u$, by $\lambda_l x + b_l \leq \text{ReLU}(x) \leq \frac{u}{u-l}(x-l)$ and composes the relaxations in a backward pass,

$$A = A_L W_L \cdots A_1 W_1, \quad \ell \leq A\mathbf{x} + \mathbf{b} \leq u, \quad (1)$$

with $A_i = \text{diag}(\lambda_i)$. Since A is linear in the original input, cross-variable dependencies survive across all layers, which is what makes CROWN much tighter than IBP. α -CROWN [27] turns the fixed slopes into optimisable per-neuron parameters $\alpha_i \in [0, 1]$ and tightens the bound by gradient ascent, without branching.

Complete verification [22] adds Branch-and-Bound: when the relaxation is too loose the problem is split, e.g. by fixing a ReLU to $x \geq 0$ or $x < 0$, and bounds are recomputed per subdomain. β -CROWN encodes the split constraints as Lagrangian dual variables β inside the propagation, so thousands of subdomains are evaluated in one batched GPU pass with α and β optimised jointly; BICCOS [18] strengthens the root bound with cutting planes from the search tree, while other complete verifiers replace the search itself, by continuous relaxation [23], clause learning [7], adaptive set representations [1] or mixed-integer encodings [21].

2.2 The auto_LirPA Library

All our work builds on `auto_LirPA` [26], the library under α, β -CROWN, which expresses bound propagation over a PyTorch computational graph and derives bounds for any intermediate node of an arbitrary architecture. It supports *batch BaB*: up to hundreds of thousands of open subproblems are bounded in a single pass. Any modification must therefore keep batched processing intact and stay compatible with the traced graph, which is what makes the sharding work non-trivial.

3 Sharding Bound Propagation

3.1 Primitives, and Why Not Pipeline Parallelism

Tensor Parallelism [19] splits the matrix products inside a layer: column-parallel splits $W \in \mathbb{R}^{k \times n}$ along the output dimension into $W^{(r)} \in \mathbb{R}^{k \times n/P}$ and each GPU computes a partial output from a replicated input, while row-parallel splits along the input dimension and sums the partial products, so a Column-Row pair costs exactly one `AllReduce`. *FSDP* [16] shards every parameter tensor across P GPUs and an `AllGather` rebuilds a layer's parameter just before it is needed,

releasing it just after, so at most one full weight is resident. *Pipeline parallelism* [9] does not fit CROWN: Equation 1 traverses *all* layers in one backward pass, so a pipeline would ship weight matrices between stages at every step, i.e. TP with a worse communication pattern. TP and FSDP both run the full pass on every GPU and differ in what they shard: TP the computation, FSDP only the storage.

3.2 Tensor Parallelism

We add two operators, `BoundLinearTP_Col` and `BoundLinearTP_Row`, derived from `BoundLinear`. Each calls `AllReduce` in `bound_backward`. Each also has a custom ONNX symbolic method, so JIT tracing builds the graph without running the collective; otherwise tracing fails with `Tried to trace ProcessGroup`. The function `tp_shard_bounded_module` works on any `BoundedModule`, including one loaded from ONNX. It replaces pairs of consecutive `BoundLinear` nodes with their TP versions and shards the weights. A BFS pass then marks each Column–Row subgraph as a *sharded zone*, so that its intermediate bounds are handled the same way on every rank.

TP shards W and the A -matrices alike, so the backward step $A \leftarrow A \cdot A_i W_i$ stays local inside a zone, and one `AllReduce` at the Row layer restores the exact result. The ReLU between the Column and the Row layer needs bounds on the Column outputs to pick its slopes λ_i . Computing them by CROWN would need a separate backward pass with communication at every step, which would give the memory back, so each rank bounds its own Column outputs by IBP. In the first zone this costs nothing: the Column layer is a single affine map of the input box, and IBP is exact there. In every later zone the Column input is already an interval, and IBP discards the correlation between its components. This is the *wrapping effect* [13].

How fast tightness degrades. Let IBP pass through k linear layers with ReLUs in between, and let $w(\cdot)$ denote elementwise interval width. A linear layer maps widths by $w(W\mathbf{x}) \leq |W| w(\mathbf{x})$ componentwise, and ReLU is non-expansive, $w(\text{ReLU}(x)) \leq w(x)$. Composing gives

$$\|w_{\text{out}}\|_{\infty} \leq \left(\prod_{i=1}^k \|W_i\|_{\infty} \right) \|w_{\text{in}}\|_{\infty}. \quad (2)$$

This is an upper bound on *IBP* widths, not a growth rate of the CROWN–IBP gap, and it is the only claim we make: the gap is governed by the product of layer norms, so it compounds per zone. The measured behaviour (Table 2) is consistent with it, and $\|W\|_{\infty}$ for the three MNIST-FC models is 19.6, 20.4 and 21.1. The mechanism is elementary: for $x_1, x_2 \in [-1, 1]$ and $W = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$, CROWN gives $y_1 + y_2 = 2x_1 \in [-2, 2]$ exactly, while IBP treats $y_1, y_2 \in [-2, 2]$ as independent and returns $[-4, 4]$.

3.3 Fully Sharded Data Parallelism

`fsdp_shard_bounded_module` replaces every `BoundParams` weight with its shard along `dim=0`. For `BoundLinear` the shard is $W^{(r)} \in \mathbb{R}^{k/P \times n}$, and for `BoundConv` it is $W^{(r)} \in \mathbb{R}^{C_{\text{out}}/P \times C_{\text{in}} \times k_H \times k_W}$. In both layer types dimension 0 is the output axis, so an `AllGather` along it gives back the exact matrix. A tensor stays unsharded if its first dimension is not divisible by P ; at $P=4$ this happens only for the classifier layer. Two hooks, `fsdp_gather_node` and `fsdp_free_node`, run inside `backward_general` (CROWN) and `IBP_general` (IBP). They build the full matrix right before a layer needs it and free it right after. The code is about 100 lines, against about 500 for TP, because it needs no new operators and leaves the graph as it is.

FSDP performs the same floating-point operations in the same order as the single-GPU path; only the provenance of the weight tensor differs. IBP and CROWN bounds are therefore *bitwise* identical — an exact zero, not a machine epsilon — which we confirm on eight configurations (MLPs of width 256 at depths 2, 4, 6 under IBP and CROWN, and ONNX 256×2 and 256×4 under CROWN) at $P=2$ and $P=4$. We report it as a regression guarantee rather than as a result, since it follows from the construction.

FSDP also composes with complete verification without extra algorithmic machinery: each rank runs an ordinary β -CROWN+BaB iteration once its weights have been gathered, and β is managed exactly as on one GPU. Sharding `BoundConv` over output channels required three files to change (`fsdp_utils.py` to admit `BoundConv` and to prevent double sharding, `backward_bound.py` for the gather/free in the CROWN backward, `interval_bound.py` for the free after IBP). Transformers needed three JIT fixes, because `auto_LiRPA` bakes reshape arguments in at batch size 1 while BaB later increases the batch: `BoundReshape` substitutes the actual batch size when $\prod(\text{new_shape}) = x.\text{numel}()$, `BoundConcat` takes its tensor count from the trace, and `BoundConstantOfShape` updates its shape argument.

4 Where the Memory Actually Is

We read GPU memory from the `torch.cuda` counters and split it in two. The *resident* part R is allocated before `compute_bounds` starts. It holds the weights, the traced graph and the inputs. The *transient* part $T = M_{\text{peak}} - R$ appears during the pass and holds A -matrices, intermediate bounds, gathered weights and collective buffers. Sharding can only reduce R .

For an MLP with n inputs and d hidden layers of width h , the weights take $S_W = 4(nh + (d-1)h^2 + hm)$ bytes. The A -matrices are harder to count, because CROWN builds a new one for every layer whose intermediate bounds it needs. For a layer of width h the specification dimension is h , so with N inputs each A -tensor has shape $[N, h, h]$ and a lower/upper pair takes $8Nh^2$ bytes. With c pairs alive at the same time we get

$$M_{\text{peak}} \approx S_W + 8cNh^2 + S_0, \quad (3)$$

where S_0 is the rest: graph metadata, inputs and intermediate bounds. To find c we logged the allocated memory after every node of a CROWN pass at $h=4096$, $d=4$, $N=1$. It went from 417 to 1844 MB, and most of the increments were exactly 128 MB, the size of one A -pair of shape $[4096, 1, 4096]$. When the pass finished, 18 such tensors were still alive. That gives $c=9$ and 1152 MB of A -matrices next to 204.5 MB of weights. With more inputs or a wider output specification the A -matrices grow further, while the weights stay the same.

Equation 3 gives a ceiling for FSDP. Sharding turns S_W into S_W/P and does not touch the other terms, so no implementation can save more than

$$\eta_{\max} = \left(1 - \frac{1}{P}\right) \frac{S_W}{M_{\text{peak}}}. \quad (4)$$

For our MLPs $\eta_{\max}$ is 5.3% at $h=4096$, $d=4$ and 5.2% at $h=8192$. It drops to 2.9% at $d=8$ and rises to 7.9% at $P=4$. A real implementation also pays for the gathered weight and the NCCL buffers, and Section 5.4 shows that these costs are larger than the saving.

In complete verification the axis is the subdomain count. In β -CROWN+BaB the largest tensors are indexed by the number B of subdomains in the batch. This holds for the per-neuron α slopes, whose shape is $(\text{layers}) \times (\text{specs}) \times (B) \times (\text{neurons})$, and also for the intermediate bounds and the A -matrices. The weights do not depend on B . We swept B from 128 to 4096 on `mnist-net_256x6` (Table 5) and fitted a line through the six points:

$$M_{\text{peak}}(B) \approx 21.3 + 0.372 B \quad \text{MB}, \quad (5)$$

Each batch-dependent term doubles when B doubles, while the weights stay at 2.03 MB, which is 0.13% of the peak at $B=4096$. One might expect the α tensors to dominate, but at $B=4096$ they take only 121 MB (7.9% of the peak). Intermediate bounds and A -matrices take 290 MB each (18.8%). We could not attribute the remaining $\approx 54\%$ to a tensor class, but it also grows linearly with B , so the axis worth sharding is B .

5 Experiments

Setup. Unless stated otherwise, runs used $4 \times$ NVIDIA RTX PRO 4000 Blackwell (24 GB, PCIe, no NVLink, driver 580.159.04), 48 vCPU, PyTorch 2.8.0+cu128, CUDA 12.8 and NCCL with `NCCL_P2P_DISABLE=1`, launched by `torchrun`, with seed 42, one discarded warm-up and three measured repeats. Each mode runs in its own process. Under FSDP the returned bound tensors keep an autograd graph alive, and in a shared process this memory would be counted in the peak of the next measurement. The ViT and CIFAR-100 checks of Section 5.6 ran on $2 \times$ NVIDIA A40 (48 GB). We also measured the MLP configurations on the A40, and the peaks agree to four significant digits, since allocated memory depends only on tensor shapes.

Table 1. TP scaling. Peak memory and propagation time both scale close to linearly. Comm is the time inside `AllReduce`, including graph construction and waiting for the slower rank. The single-GPU peak at $H=131\,072$ equals the TP=2 per-rank peak at $H=262\,144$ measured on the A40, so the TP memory model is exact.

Mode	Peak (MB)	Reduction	Build (ms)	Propagation (ms)	Speed-up	Comm (ms)
Single	13 513.2	1.00×	4 279.5	546.6	1.00×	0.0
TP=2	6 856.7	1.97×	22 679.8	287.7	1.90×	89.2
TP=4	3 528.4	3.83×	11 392.1	156.5	3.49×	139.9

Table 2. TP=2 deviation from the single-GPU reference. A single zone fed by the input box agrees to machine precision; each further zone compounds the IBP substitution, consistently with Equation 2. Soundness holds throughout, and cross-rank deviation is zero everywhere.

Model	Zones	$\ W\ _\infty$	$ \Delta lb $	$ \Delta ub $	Sound
PyTorch 256×2	1	—	3×10^{-8}	3×10^{-8}	✓
PyTorch 256×4	2	—	2.51	2.49	✓
ONNX 256×2	1	19.6	6×10^{-8}	6×10^{-8}	✓
ONNX 256×4	2	20.4	5.64	3.98	✓
ONNX 256×6	3	21.1	792	750	✓

5.1 Tensor Parallelism: Memory and Time

The model is $f(\mathbf{x}) = W_2 \text{ReLU}(W_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$ with W_1 Column-parallel and W_2 Row-parallel following the Megatron-LM pattern [19]; $D=4096$, $H=131\,072$, $N=2048$, $\varepsilon=0.01$, CROWN; H is chosen so that the single-GPU baseline fits in 24 GB. The custom TP operators make JIT tracing much slower, but this cost is paid once per model, not once per query, so we time graph construction separately. Timed together, TP looks $4.8\times$ slower than the baseline, although its propagation is faster.

Table 1 is the strongest positive result in this paper. The model has a single zone fed directly by the input box, so the IBP fallback of Section 3.2 is exact here. We measure tightness on smaller models of the same structure in Section 5.2, where one zone agrees with the single-GPU bounds to machine precision.

5.2 Tensor Parallelism: Soundness and Tightness

The reference is single-GPU CROWN. We run it and TP=2 on PyTorch MLPs and on the VNN-COMP 2022 MNIST-FC ONNX models [15] ($\varepsilon=0.02$, L_∞). For each model we look at $|\Delta lb|$ and $|\Delta ub|$, at one-sided soundness ($lb_{\text{tp}} \leq lb_{\text{ref}}$, $ub_{\text{tp}} \geq ub_{\text{ref}}$) and at whether the ranks agree.

The same holds for α -CROWN. Compared with a single-GPU α -CROWN run, TP=2 differs by at most $|\Delta lb|_{\text{max}} = 4.47 \times 10^{-8}$ (mean 2.14×10^{-8} , relative 3.64×10^{-7}), well within the tolerance of 10^{-4} . We applied `tp_shard_bounded_module` to all three MNIST-FC ONNX models without manual changes. It walks

Table 3. FSDP against single-GPU, one mode per process, after the two memory fixes described below. Weight shards are $1/P$ of the weights while peak memory rises; $P=4$ is indistinguishable from $P=2$ because the term it shards is not the term that dominates.

Model	Weight tensors (MB)			Peak (MB)			
	single FSDP=2 offload			single FSDP=2	FSDP=4	offload	
$h=1024, d=4$	15.1	7.6	0	149.6	200.6	200.6	184.6
$h=4096, d=4$	204.5	102.3	0	1 944.9	2 476.0	2 475.9	2 271.6
$h=8192, d=4$	792.9	396.5	0	7 593.9	9 551.8	9 551.5	8 759.0
$h=4096, d=8$	460.5	230.3	0	7 934.0	10 181.1	10 180.9	9 720.7

the graph in topological order, replaces the **BoundLinear** pairs, reruns a forward pass to refresh the shapes and marks the zones. All three models pass the checks above.

5.3 FSDP: Exactness, and Its Limits

On eight configurations (MLPs of width 256 at depths 2, 4, 6 under IBP and CROWN; ONNX 256×2 and 256×4 under CROWN) FSDP reproduces the single-GPU bounds with $|\Delta lb| = |\Delta ub| = 0.00$, an exact zero, at both $P=2$ and $P=4$. One qualification is due. In the *optimised* α/β path a distributed run does not match a plain single-GPU run bit for bit: on `mnist-net_256x6` at $\varepsilon=0.05$, three of nine specification bounds differ, by at most 6.10×10^{-5} absolute and 1.17×10^{-7} relative. Sharding is not the cause. Our distributed launcher disables the TorchScript fusers, a workaround for a strides assertion that fires only on non-primary GPUs, and reproducing exactly that fuser configuration on a single GPU with no process group at all reproduces the distributed bounds digit for digit, while the untouched single-GPU run keeps its own values. The deviation is the price of the fuser workaround, not of weight sharding, so FSDP is exact in the sense we claimed — but a distributed run of α, β -CROWN is not bit-identical to a single-GPU one for reasons that have nothing to do with sharding, and that is worth knowing for anyone comparing multi-GPU verification results.

5.4 FSDP: Memory and Time

The weight shards are exactly $1/P$ of the weights (Table 3), but peak memory rises by 26–34%, and the ceiling of Equation 4 explains why: at $h=4096, d=4$ the whole weight term is 10.5% of the peak, so $P=2$ could have saved at most 5.3% of it, less than the cost of staging a full weight. Going from $P=2$ to $P=4$ moves the peak by 0.1 MB out of 2 476 while adding 50 ms of collectives. These numbers come from our FSDP code after two fixes. One removed a second full copy of the weight that every gather allocated, which lowered the peak by about 2%. The other made the gathered weights free after the pass. A shard, however, is still a view of the full weight tensor, so the full storage stays allocated. Releasing

Table 4. Bound-propagation time (ms). Parentheses: for FSDP, time inside **AllGather** (ms); for offloading, staged host-to-device volume (MB), whose copy time is contained in the total. The naive baseline is cheaper than sharding on every configuration.

Model	single	FSDP=2	FSDP=4	offload
$h=1024, d=4$	23.6	30.1 (17.3)	54.6 (43.7)	21.5 (51 MB)
$h=4096, d=4$	74.7	236.6 (166.7)	286.7 (217.4)	103.3 (637 MB)
$h=8192, d=4$	418.1	983.3 (583.5)	1 128.1 (733.7)	528.4 (2 427 MB)
$h=4096, d=8$	381.1	867.4 (510.8)	1 003.0 (653.4)	477.6 (2 350 MB)

it would lower each FSDP peak by $(1 - 1/P)S_W$, which is less than the gap to the single-GPU peak in every row of Table 3.

Propagation is 2.3–3.2 $\times$ slower under FSDP=2 at $h \geq 4096$ (1.3 $\times$ at $h=1024$, the smallest workload), and 57–70% of the wall-clock is spent inside **AllGather**: 166.7 of 236.6 ms at $h=4096$, 583.5 of 983.3 ms at $h=8192$. On a PCIe machine without NVLink that overhead is structural, since one full weight per layer crosses the bus at every intermediate-bound computation — 19 collectives and 842 MB moved for a $d=4$ pass at $h=4096$.

The naive baseline is better. A simpler alternative to sharding is to keep the weights in host memory and stage them per layer. We implemented this baseline by reusing the same hook sites with a host-to-device copy in place of the collective, which leaves *no* weight bytes resident on the GPU — a strictly larger reduction than FSDP’s $1/P$ — and produces bounds identical to the single-GPU reference. It wins on both axes: at $h=4096, d=4$ it needs 2 271.6 MB against FSDP’s 2 476.0 and 103.3 ms against 236.6, and the pattern holds at every size (Tables 3, 4). Activation checkpointing, the other naive alternative, does not apply at all: the CROWN backward stores no forward activations to recompute, and its A -matrices are live because they are still needed.

That the crude mechanism beats the sophisticated one is the clearest statement of this paper’s finding. Neither mechanism can win, because both pay to stage a full weight into a pass whose peak is set by tensors they never touch: at $h=4096$ the staged copies that **auto_LiRPA** retains past the layer that needs them cost 531 MB, more than twice the 204.5 MB of weights either mechanism removes.

5.5 Complete Verification: the Memory Is in the Subdomains

We use **mnist-net_256x6** [15] with $\varepsilon=0.05$ and run exactly 10 BaB rounds. The flag **bab.force_synchronous** makes the single-GPU and FSDP runs do the same work. It fixes the batch and turns off batch enlargement, early stopping and pruning in iteration.

Two things follow. First, FSDP cannot help here and does not: at $B=512$ the peak is 211.9 MB on one GPU and 214.9 MB per rank at both $P=2$ and $P=4$, because there are 2.03 MB of weights to shard and the **AllGather** buffers cost more than that.

Table 5. Where BaB memory goes, by number of subdomains B (single GPU, MB). Every batch-scaled term doubles with B ; the weight term does not move.

B	α	β	Interm. bounds	A -matrices	Weights	Peak
128	3.79	0.01	9.06	9.08	2.03	68.73
256	7.58	0.02	18.13	18.16	2.03	116.24
512	15.16	0.05	36.26	36.33	2.03	211.93
1024	30.31	0.10	72.51	72.66	2.03	402.52
2048	60.62	0.20	145.03	145.32	2.03	780.72
4096	121.25	0.40	290.06	290.64	2.03	1543.61

Second, Equation 5 makes the pay-off of sharding the subdomain axis calculable instead of speculative. Splitting a global batch B across P GPUs leaves each with B/P subdomains and hence $21.3 + 0.372 B/P$ MB, i.e. 45.0% less memory per GPU at $P=2$ and 67.5% less at $P=4$ for $B=512$ (49.3% and 73.9% at $B=4096$). The per-GPU workload of the split is a single-GPU run at B/P , which Table 5 measures directly; what the projection leaves out is the cost of gathering the per-rank results. Since B is a tunable parameter, the law also reads the other way: at the same memory per GPU, P GPUs bound P times more subdomains per round. This throughput gain is the practical pay-off.

Our implementation of the split is partial. If the domains are scattered before `build_history_and_set_bounds`, each rank builds its own state and propagates its own chunk correctly. Domain parallelism does not compose with FSDP: once ranks hold different subdomains, their per-layer `AllGather` sequences diverge and NCCL deadlocks. We therefore disable weight sharding in this mode, which costs nothing here, the weights being 2.03 MB. The gather of the per-rank results still stalls, so we report the projection rather than a measurement.

5.6 Convolutions and Transformers: Correctness Checks

These runs show that the sharding code also works beyond MLPs. We treat them as correctness checks only, since memory changes by at most 3% in either direction. On the VNN-COMP’23 ViT `pgd_2_3_16` (2 blocks, 3 heads, 0.3 MB of weights) [4], after the three JIT fixes of Section 3.3, FSDP=2 completes 78 BaB domains over 10 rounds at 1020.1 MB per rank with $|\Delta lb| = 0.00$ at printed precision — to our knowledge the first Transformer run through BaB in α, β -CROWN with sharded weights. On CIFAR-100 ResNets [3] with `BoundConv` sharding (batch 64, `max_iterations` 5), ResNet-med runs 130 domains over 5 rounds at 930 MB per rank against 960 MB on one GPU and exhausts its budget (*unknown*), while ResNet-large (≈ 95 MB of weights) is proved **unsat** by α -CROWN at initialisation at 4641.5 MB per rank against 4610.6 MB on one GPU. That *unsat* is a correctness check, not a benefit: no BaB ran, and the FSDP peak is the higher of the two, its `AllGather` overhead exceeding what sharding 95 MB of weights returns when the α tensors alone exceed 4 GB.

Table 6. Optimiser and search settings. All configuration files are in the repository; the fair-comparison configs additionally set `force_synchronous`, `auto_enlarge_batch_size`: `false`, `pruning_in_iteration`: `false` and `pgd_order`: `skip`.

Setting	MNIST-FC 256×6	ViT pgd_2_3_16	CIFAR-100 ResNet
α -CROWN lr / iters	0.1 / 20	0.5 / 50	0.25 / 20
β -CROWN lr _{α}	0.1	0.05	0.1
β -CROWN lr _{β} / iters	0.03 / 20	0.05 / 10	0.2 / 10
Batch	128–4096	32	64
BaB max_iterations	10	10	5
Branching	default	reduceop: min	kfsb, 7 cand.
ε	0.05	benchmark	0.0039

5.7 Reproducibility

All runs use seed 42. Each memory figure is the mean of three repeats after one warm-up. Its standard deviation was 0.00 MB in every configuration, so the tables have no error columns. Timings vary by less than 5% at $h \geq 4096$. At $h=1024$ a pass takes only tens of milliseconds, and the jitter reaches 24% on one GPU and 43% under FSDP=4. The scripts for every table and the collated measurements (`experiments/analysis/RESULTS.md`) are in the repository.

6 Related Work

Bound propagation. CROWN [28] introduced the backward linear relaxation, α -CROWN [27] and β -CROWN [22] added optimisable slopes and dual encoding of branching constraints, DeepPoly [20] works in a polyhedral abstract domain and OSIP [10] optimises symbolic interval propagation.

Parallel and multi-GPU verification. Earlier parallel verifiers follow two lines, and neither deals with the memory of the parameters. The first splits the search space. Marabou’s Split-and-Conquer [24] partitions the input domain across CPU workers with iterative deepening, and β -CROWN [22] batches subdomains on one GPU. The second moves a fixed abstract domain to the GPU, as GPUPoly [14] does for DeepPoly. Both scale the *number* of subproblems, while we looked at the memory of one. Our measurements show that this memory sits in the subproblem batch, so splitting domains and saving memory turn out to be the same problem.

Sharding for training. Megatron-LM [19] introduced intra-layer tensor parallelism, ZeRO [16] shards parameters, gradients and optimiser states, and GPipe [9] pipelines layer groups with micro-batches. To our knowledge neither had been applied to bound-propagation verification, which is what makes the negative result worth recording: a training step is dominated by parameters and optimiser state, a verification pass by tensors indexed by specifications and subdomains.

7 Conclusion

Both sharding schemes were easy to add to α, β -CROWN, but they behave very differently. TP shards the specification axis, so peak memory drops by $1.97\times$ on 2 GPUs and $3.83\times$ on 4, and propagation gets $1.90\times$ and $3.49\times$ faster. Its bounds stay exact in the first sharded zone and loosen in later ones, where IBP is used. FSDP keeps IBP and CROWN bounds bitwise identical, yet it adds 26–34% to peak memory and up to $3.2\times$ to propagation time, and staging weights from host memory does better on both. The weights are only 6–11% of the peak, so sharding half of them can save a few percent at most.

The finding we would carry forward is the measurement, not either mechanism. In incomplete verification A -matrices exceed weights by $5.6\times$ at $h=4096$; in complete verification peak memory is $21.3 + 0.372 B$ MB in the number of subdomains, of which weights are 0.13%. Multi-GPU verification should therefore shard the subdomain axis, for which we project 45.0% and 67.5% less memory per GPU at $P=2, 4$, or P times more subdomains per round at the same memory. The scatter already runs correctly per rank; the remaining task is the gather of per-rank results. Sharding the specification axis without the IBP fallback — exact intermediate bounds inside later sharded zones — is the harder and more valuable open problem.

Acknowledgments. The authors thank the `auto_LirPA` / α, β -CROWN team for the open-source codebase that made this work possible, and the anonymous reviewers, whose comments improved the memory analysis.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Bak, S.: nnenum: Verification of ReLU neural networks with optimized abstraction refinement (2021), https://www.researchgate.net/publication/351683889_nnenum_Verification_of_ReLU_Neural_Networks_with_Optimized_Abstraction_Refinement
2. Bertschinger, D., Hertrich, C., Jungeblut, P., Miltzow, T., Weber, S.: Training neural networks is $\exists\mathbb{R}$ -complete. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 34 (2021), https://proceedings.neurips.cc/paper_files/paper/2021/file/9813b270ed0288e7c0388f0fd4ec68f5-Paper.pdf
3. Brix, C., Bak, S., Johnson, T.T., Wu, H.: The fifth international verification of neural networks competition (VNN-COMP 2024): Summary and results (2024), <https://arxiv.org/abs/2412.19985>
4. Brix, C., Bak, S., Liu, C., Johnson, T.T.: The fourth international verification of neural networks competition (VNN-COMP 2023): Summary and results (2023), <https://arxiv.org/abs/2312.16760>
5. Demarchi, S., Guidotti, D., Pulina, L., Tacchella, A.: VNN-LIB: The verification of neural networks library (2021), <https://www.vnnlib.org/>

6. Demarchi, S., Guidotti, D., Pulina, L., Tacchella, A.: Supporting standardization of neural networks verification with VNN-LIB and CoCoNET. In: Proceedings of the 6th Workshop on Formal Methods for ML-Enabled Autonomous Systems (FoMLAS 2023). Kalpa Publications in Computing, vol. 16, pp. 47–58 (2023), <https://easychair.org/publications/paper/Qgdn/open>
7. Duong, H., Nguyen, T., Dwyer, M.: NeuralSAT: A DPLL(T)-based framework for verifying deep neural networks (2023), <https://github.com/dynaroars/neuralsat>
8. Guidotti, D.: Enhancing neural networks through formal verification. In: Proceedings of the AI*IA 2019 Doctoral Consortium (AI*IA 2019 DC). CEUR Workshop Proceedings, vol. 2495, pp. 107–112. CEUR-WS.org (2019), <https://ceur-ws.org/Vol-2495/paper13.pdf>
9. Huang, Y., Cheng, Y., Bapna, A., Firat, O., Chen, D., Chen, M.X., Lee, H., Ngiam, J., Le, Q.V., Wu, Y., Chen, Z.: GPipe: Efficient training of giant neural networks using pipeline parallelism. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 32 (2019), <https://arxiv.org/abs/1811.06965>
10. Kern, P., Kleine Büning, M., Sinz, C.: Optimized symbolic interval propagation for neural network verification (2022), <https://arxiv.org/abs/2212.08567>
11. Liu, J., Chen, L., Miné, A., Wang, J.: Input validation for neural networks via runtime local robustness verification. CoRR **abs/2002.03339** (2020), <https://arxiv.org/abs/2002.03339>
12. Marzari, L., Mastroeni, I., Farinelli, A.: Advancing neural network verification through hierarchical safety abstract interpretation (2025), <https://arxiv.org/abs/2505.05235>
13. Moore, R.E., Kearfott, R.B., Cloud, M.J.: Introduction to Interval Analysis. SIAM (2009)
14. Müller, C., Makarchuk, G., Singh, G., Püschel, M., Vechev, M.: Scaling polyhedral neural network verification on GPUs (2021), <https://files.sri.inf.ethz.ch/website/papers/mler2021neural.pdf>
15. Müller, M.N., Brix, C., Bak, S., Liu, C., Johnson, T.T.: The third international verification of neural networks competition (VNN-COMP 2022): Summary and results (2022), <https://arxiv.org/abs/2212.10376>
16. Rajbhandari, S., Rasley, J., Ruwase, O., He, Y.: ZeRO: Memory optimizations toward training trillion parameter models (2020), <https://arxiv.org/abs/1910.02054>
17. Ruan, W., Huang, X., Kwiatkowska, M.: Reachability analysis of deep neural networks with provable guarantees. In: Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence (IJCAI) (2018), <https://www.ijcai.org/proceedings/2018/0368.pdf>
18. Shi, Z., Jin, Q., Kolter, J.Z., Jana, S., Hsieh, C.J., Zhang, H.: Scalable neural network verification with branch-and-bound inferred cutting planes. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 37 (2024), https://proceedings.neurips.cc/paper_files/paper/2024/file/33d93e4dc57453e7667b20f62e7c0681-Paper-Conference.pdf
19. Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., Catanzaro, B.: Megatron-LM: Training multi-billion parameter language models using model parallelism (2020), <https://arxiv.org/abs/1909.08053>
20. Singh, G., Gehr, T., Püschel, M., Vechev, M.: An abstract domain for certifying neural networks (2019), <https://ggndpsngh.github.io/files/DeepPoly.pdf>
21. Tjeng, V., Xiao, K., Tedrake, R.: Evaluating robustness of neural networks with mixed integer programming (2017), <https://arxiv.org/abs/1711.07356>

22. Wang, S., Zhang, H., Xu, K., Lin, X., Jana, S., Hsieh, C.J., Kolter, J.Z.: Beta-CROWN: Efficient bound propagation with per-neuron split constraints for neural network robustness verification. In: *Advances in Neural Information Processing Systems (NeurIPS)* (2021), <https://arxiv.org/abs/2103.06624>
23. Wu, H., Isac, O., Zeljić, A., Tagomori, T., Daggitt, M., Kokke, W., Refaeli, I., Amir, G., Julian, K., Bassan, S., Huang, P., Lahav, O., Wu, M., Zhang, M., Komendantskaya, E., Katz, G., Barrett, C.: Marabou 2.0: A versatile formal analyzer of neural networks (2024), <https://arxiv.org/html/2401.14461v1>
24. Wu, H., Ozdemir, A., Zeljić, A., Julian, K., Irfan, A., Gopinath, D., Fouladi, S., Katz, G., Păsăreanu, C., Barrett, C.: Parallelization techniques for verifying neural networks. In: *Formal Methods in Computer Aided Design (FMCAD)*. pp. 128–137 (2020)
25. Wurm, A.: Robustness verification in neural networks (2024), <https://arxiv.org/abs/2403.13441>
26. Xu, K., Shi, Z., Zhang, H., Wang, Y., Chang, K.W., Huang, M., Kailkhura, B., Lin, X., Hsieh, C.J.: Automatic perturbation analysis for scalable certified robustness and beyond (2020), https://github.com/Verified-Intelligence/auto_LiRPA
27. Zhang, H., Wang, S., Xu, K., Li, L., Li, B., Jana, S., Hsieh, C.J., Kolter, J.Z.: alpha-beta-CROWN: A complete and efficient bound propagation based neural network verifier (VNN-COMP 2021–2024) (2023), https://github.com/Verified-Intelligence/alpha-beta-CROWN_vnncomp23
28. Zhang, H., Weng, T.W., Chen, P.Y., Hsieh, C.J., Daniel, L.: Efficient neural network robustness certification with general activation functions (2018), <https://arxiv.org/abs/1811.00866>